autolisp-reader Specification

Codex

May 25, 2026

Contents

1 Overview

This document specifies the first reader subsystem for the `clautolisp` implementation subproject.

The reader is responsible for:

- accepting text input from files, strings, and streams,
- decoding text at the input boundary,
- normalizing line endings for syntax processing,
- tokenizing and parsing AutoLISP source,
- representing parsed objects and optionally comments as source-aware data,
- distinguishing strict and lax acceptance modes,
- separating AutoLISP syntax from Common Lisp reader behavior.

This document is intentionally narrower than the repository-wide `AutoLISP Spec`. It is a module specification for one implementation subsystem.

2 Scope

The `autolisp-reader` system covers:

- lexical syntax for core AutoLISP forms,
- source position tracking,
- strict and lax reader acceptance policies,
- source-aware representations for literal objects and read forms,
- optional comment retention for tooling.

The `autolisp-reader` system does not define:

- evaluation semantics,
- symbol value/function binding semantics,
- printer semantics except where round-trippable source spelling matters,
- host interaction semantics.

3 Definitions

3.1 Reader Boundary

The reader boundary is the stage that converts external text input into a normalized character stream for tokenization.

This boundary is responsible for:

- external-format decoding,
- line-ending normalization,
- preserving enough source mapping information to report original locations accurately.

3.2 Strict Mode

Strict mode is a token-acceptance policy that rejects lexical forms that are under-specified, weakly portable, ambiguous, or only tolerated for compatibility.

3.3 Lax Mode

Lax mode is a token-acceptance policy that accepts broader real-world token syntax when doing so is useful for compatibility and migration.

Portable syntax shared across supported implementations and versions belongs to the common acceptance core and shall therefore be accepted in both strict mode and lax mode.

3.4 Reader Options

Reader options are orthogonal controls that affect how the reader reports or preserves syntax without changing the strict versus lax token-acceptance policy.

Initial reader options include:

- whether comments are skipped or retained,
- whether malformed structure is reported immediately or preserved as recoverable syntax for tooling-oriented entry points,
- whether integer-shaped decimal tokens that overflow the signed 32-bit integer range produce a warning when they are classified as reals.

3.5 Reader Object

A reader object is a source-aware representation produced by the reader.

Reader objects are not raw Common Lisp symbols or numbers used directly as the semantic AutoLISP runtime objects.

4 Normative Rules

4.1 Input Normalization

- The reader shall accept input from strings, character streams, and byte-oriented sources.
- Byte-oriented sources shall be decoded before tokenization.
- Line endings `LF`, `CRLF`, and `CR` shall all be accepted as input.
- The parser shall operate on a normalized newline model.

- Source locations shall preserve enough information to report file, line, and column positions consistently even when line endings were normalized.

4.2 Top-Level Syntax

- The reader shall accept zero or more top-level forms separated by whitespace or comments.
- A top-level form may be an atom, a quoted form, a list, or a dotted pair inside list notation.
- Unexpected end of input inside a string literal or list shall be a syntax error.
- A dot token used outside dotted-pair position shall be a syntax error in strict mode.
- A tooling-oriented recovery option may preserve such malformed input as a recoverable syntax node, but normal batch reading shall still report it as an error.

4.3 Delimiters

- Left parenthesis `(` starts a list.
- Right parenthesis `)` ends a list.
- Apostrophe `'` introduces the quote shorthand and shall read as a quoted form node rather than as part of a symbol token.
- Semicolon `;` starts a line comment and continues through the end of the physical source line.
- Double quote `\"` starts and ends a string literal.

4.4 Whitespace

- Space, tab, form feed, and normalized newline characters are reader whitespace.
- Whitespace separates tokens but is otherwise not represented in normal reader output.
- Whitespace spans may be retained in auxiliary source maps, but they are not first-class reader objects in the initial design.

4.5 Comments

- A semicolon comment begins at `;` and ends immediately before the next physical line terminator or end of input.
- By default, comments are skipped and do not appear in the ordinary parsed form stream.
- The reader shall provide an option to retain comments as source-aware comment objects.
- When comments are retained, their relative ordering with neighboring forms shall be preserved.
- Comment retention changes the concrete syntax stream seen by downstream consumers.
- Therefore a comment-preserving reader result shall not be modeled as the same object stream as ordinary comment-skipping form reading.
- In particular, comments that occur around dotted-pair syntax must not be forced into ordinary cons structure in a way that destroys the syntactic role of the dot token.

4.6 Lists and Dotted Pairs

- `()` reads as the empty list object.
- `(a b c)` reads as a proper list of three elements.
- `(a . b)` reads as a dotted pair.
- Exactly one dot in cdr position is accepted for dotted-pair syntax in strict mode.
- Repeated dots, misplaced dots, or extra elements after a dotted cdr marker are syntax errors.
- A tooling-oriented recovery option may record some malformed dotted constructs as recoverable syntax objects when an appropriate entry point explicitly asks for recovery.

4.7 Strings

- Strings are delimited by double quotes.
- The semantic string value is an ordered sequence of characters.
- The reader shall preserve the raw source spelling of the string literal when source retention is enabled.
- The exact escape repertoire is under-specified by the vendor corpus and shall therefore be represented by an explicit acceptance policy.
- In the initial baseline accepted by both strict and lax modes, the reader shall accept doubled double quote, `\"`, and `\\` inside strings.
- This baseline is an implementation choice motivated by real-world corpus needs and by the plausible inference that products may defer to a C-like escape model for at least these minimal cases.
- In strict mode, unknown backslash escapes shall not be given implicit Common Lisp meaning.
- Additional implementation-defined escape acceptance, such as `\u+00E9`, may be enabled by separate reader options and shall be kept distinct from strict versus lax token mode.

4.8 Symbols

- A symbol token is any non-delimiter, non-whitespace token that is not recognized as another literal token class.
- Symbol names shall be represented independently of Common Lisp package symbols.
- The reader shall preserve the original source spelling of symbol tokens.
- The reader shall also provide a canonicalized symbol name used for later runtime interning.
- The initial canonicalization rule is case-insensitive folding to a single implementation-defined canonical case.
- The original spelling and the canonical name must both remain available in the reader result.
- Characters that are token delimiters shall not appear unescaped inside strict symbol tokens.

4.9 Integer Literals

- Strict integer syntax is: optional sign followed by one or more decimal digits.
- In regexp form, the strict grammar is `[+-]?[0-9]+`.
- Strict integer syntax does not admit embedded whitespace, radix prefixes, decimal points, or exponent markers.
- A token matching this grammar is classified as an integer object only when its numeric value lies within the signed 32-bit range `[-2147483648, 2147483647]`.
- A token matching this grammar whose numeric value lies outside that range shall be classified as a real object instead.
- The reader shall provide an option to warn when such integer-shaped overflow tokens are classified as reals.
- Lax integer syntax may accept additional spellings when a dialect profile explicitly requests them.
- A token accepted as an integer in lax mode shall retain its original lexeme so later stages can distinguish strict versus lax acceptance.

4.10 Real Literals

- Strict real syntax is decimal-only and requires either:
  - digits, decimal point, digits, optionally followed by exponent, or
  - sign plus the same form.
- In regexp form, the initial strict grammar is `[+-]?[0-9]+\\.[0-9]+([eE][+-]?[0-9]+)?`.
- Strict real syntax therefore rejects `.5`, `1.`, and exponent-only integer spellings such as `1e3`.
- This is intentionally conservative because vendor documentation does not yet define a stable portable lexical grammar for all reader and conversion cases.
- Lax real syntax may additionally accept:
  - `.5`,

  - `1.`,
  - exponent notation without a fractional part such as `1e3`,
  - other explicitly enabled implementation-defined spellings.

- Hexadecimal floating syntax shall be rejected in strict mode.
- Hexadecimal floating syntax may only be accepted in lax mode by an explicitly implementation-defined extension.
- Integer-shaped decimal tokens outside the signed 32-bit integer range are also classified as real objects even when they do not use explicit real syntax.

4.11 Quote Syntax

- `'x` shall read as a quote syntax node whose payload is the following form.
- The reader output shall preserve the distinction between explicit quote syntax and an equivalent expanded list form when source retention is enabled.
- The downstream evaluator bridge may later lower quote syntax into a canonical representation.

5 Implementation Guidance

5.1 Reader Result Model

The first implementation should produce concrete syntax objects rather than bare Common Lisp values.

Each reader result should contain at least:

- kind,
- source span,
- raw lexeme or raw slice when meaningful,
- normalized value or normalized name when meaningful,
- child objects for compound syntax,

- dialect and acceptance metadata when the parse depended on non-strict acceptance.

The module should expose at least two distinct result views:

- a form-oriented view for ordinary reading with comments skipped,
- a concrete-syntax or event view for tooling-oriented reading where comments, dot markers, and similar surface tokens may need to remain explicit.

5.2 Recommended Object Families

The following object families should exist as distinct reader-layer data structures:

5.2.1 Source Span

A source span should record:

- source designator,
- start line,
- start column,
- end line,
- end column,
- optionally byte or character offsets.

5.2.2 Comment Object

A comment object should record:

- raw comment text including or excluding the leading semicolon by explicit convention,
- source span,
- comment kind, initially only `:line-comment`.

5.2.3 Dot Object

A dot object should record:

- original token spelling,
- source span.

This object is needed in concrete-syntax or event views when the role of the dot token must remain explicit instead of being collapsed into dotted cons structure.

5.2.4 Symbol Object

A symbol object should record:

- original token spelling,
- canonical symbol name,
- source span.

It may additionally record:

- whether the token was accepted only in lax mode,
- future package/namespace annotations if the language later requires them.

5.2.5 String Object

A string object should record:

- decoded character contents,
- original token spelling,
- source span.

It may additionally record:

- whether escape processing used strict or lax rules.

5.2.6 Integer Object

An integer object should record:

- numeric value,
- original token spelling,
- source span,
- acceptance mode.

Integer objects are produced only for values in the signed 32-bit range.

5.2.7 Real Object

A real object should record:

- numeric value,
- original token spelling,
- source span,
- acceptance mode.

Real objects may arise either from explicit real syntax or from integer-shaped decimal tokens whose value exceeds the signed 32-bit integer range.

5.2.8 Cons/List Object

A cons-form object should record:

- ordered child objects,
- whether the surface syntax was proper-list or dotted-pair notation,
- source span.

5.2.9 Quote Object

A quote object should record:

- quoted object,
- source span of the quote marker and payload.

5.3 Strict/Lax Boundary Strategy

Strict mode should define the portable core grammar used by tests, specification examples, and future conformance claims.

Lax mode should:

- accept the entire strict portable core,
- accept additional syntax only behind explicit switches,
- tag that acceptance in the resulting objects,
- never silently erase the distinction between strict and lax parsing,
- support future migration work.

Recovery, comment retention, and other tooling-oriented behaviors should be modeled as separate reader options rather than as part of the strict/lax token distinction.

5.4 Comments As First-Class Syntax

Ordinary evaluation-oriented reading should skip comments.

A tooling-oriented entry point should support comment-preserving reads so that:

- formatters can round-trip comments,
- source-to-source transforms can maintain attachment order,
- editors can inspect comment placement without reparsing raw text separately.

The initial comment-preserving design should preserve comments as separate nodes in a top-level and list-local concrete syntax stream, rather than trying to attach every comment heuristically to a neighboring form.

That concrete syntax stream should preserve explicit delimiter and marker tokens when needed for correctness, including the dot token in dotted-pair notation.

Example:

```
( a ; comment 1
  . ; comment 2
  c ; comment 3
)
```

Ordinary form reading may produce the dotted pair ~ (a . c) ~.

Comment-preserving concrete reading should instead preserve an ordered stream equivalent in structure to:

```
(a #S(comment "...") #S(dot) #S(comment "...") c #S(comment "..."))
```

rather than trying to encode comments directly inside the dotted pair object stream.

6 Compatibility

6.1 Known Vendor Facts

- Autodesk documentation clearly documents semicolon comments, strings, symbols, lists, dotted pairs, and quote usage at the language level, but does not provide a complete formal reader grammar.
- The broader repository specification already distinguishes strict and lax acceptance modes for under-specified behavior.
- `atoi` and `atof` are specifically identified as lexical-boundary questions still requiring product tests.

6.2 Initial Reader Policy

- The reader specification adopts a deliberately restrictive strict numeric grammar for the portable core presently supported by evidence.
- Any syntax judged portable across supported implementations and versions belongs to that core and is therefore accepted by both strict and lax modes.
- Numeric classification is not determined by token shape alone: integer-shaped decimal tokens outside the signed 32-bit range are classified as reals.
- A separate reader option may request a warning when that reclassification occurs.
- Broader numeric spellings belong only to the lax extension space until product tests justify promoting them into the shared portable core.
- String escape handling remains intentionally conservative in strict mode because the vendor corpus does not yet provide a complete portable escape grammar.

7 Dictionary Entries

7.1 Semicolon Comment

7.1.1 Name

Semicolon comment

7.1.2 Class

Reader Syntax

7.1.3 Concrete Syntax

; followed by zero or more non-line-terminator characters.

7.1.4 Constituents

- comment introducer: ;
- comment body: arbitrary characters other than the terminating physical line ending

7.1.5 Description

Introduces a source comment extending to end of line.

7.1.6 Acceptance Rules

- Accepted in strict and lax modes.
- Ends before the next physical line terminator or end of input.

7.1.7 Strict/Lax Policy

- No difference in basic acceptance.
- Only the retention policy differs by reader option.

7.1.8 Examples

- `; comment`
- `(+ 1 2) ; trailing comment`

7.1.9 Compatibility

- Expected to be portable across Autodesk AutoLISP and BricsCAD-compatible dialects.

7.1.10 Notes

- The reader module exposes an option to retain these comments as comment objects.

7.1.11 Source Notes

- Documented: Autodesk tutorial material documents semicolon comments.
- Inferred: comment-retention objects are a `clautolisp` tooling design choice.

7.2 Symbol Token

7.2.1 Name

Symbol token

7.2.2 Class

Lexical Syntax

7.2.3 Concrete Syntax

A non-empty token that is not whitespace, not a delimiter token, and not recognized as a numeric literal or string literal.

7.2.4 Constituents

- token character sequence

7.2.5 Description

Represents a symbolic name in AutoLISP source.

7.2.6 Acceptance Rules

- Accepted in strict and lax modes subject to delimiter exclusion.
- Canonicalization is applied after token recognition.

7.2.7 Strict/Lax Policy

- Strict mode excludes under-specified token spellings that collide with delimiter or numeric syntax.
- Lax mode may accept additional compatibility spellings, but the original lexeme is preserved.

7.2.8 Examples

- `foo`
- `c:line`
- `*error*`

7.2.9 Compatibility

- Exact case and character rules remain partly under-specified in vendor materials and may require dialect notes later.

7.2.10 Notes

- Reader symbols are not Common Lisp package symbols.

7.2.11 Source Notes

- Documented: Autodesk documentation discusses symbols and variables.
- Inferred: exact lexical fallback rule and canonicalization strategy.
- Implementation-defined: chosen canonical case.

7.3 Strict Integer Literal

7.3.1 Name

Strict integer literal

7.3.2 Class

Token Class

7.3.3 Concrete Syntax

```
[+-]?[0-9]+
```

7.3.4 Constituents

- optional sign
- one or more decimal digits

7.3.5 Description

Portable decimal integer token recognized in strict mode.

7.3.6 Acceptance Rules

- Entire token must match the grammar.
- No leading whitespace inside the token.
- No radix prefix, decimal point, or exponent marker.
- Numeric value must lie within `[-2147483648, 2147483647]` for the token to produce an integer object.
- If the token matches the grammar but lies outside that range, it is instead read as a real.
- A reader option may request a warning for that reclassification.

7.3.7 Strict/Lax Policy

- Strict mode accepts only the grammar above.
- Lax mode may accept more spellings, including ones motivated by product behavior or migration support.

7.3.8 Examples

- `0`
- `17`
- `-42`

7.3.9 Compatibility

- Conservative subset intended to avoid accidental Common Lisp numeric syntax leakage.

7.3.10 Notes

- This entry is separate from the behavior of string conversion functions such as `atoi`.
- Token shape and numeric classification are distinct: some integer-shaped tokens produce real objects because of range overflow.
- Warning on integer-overflow reclassification is an orthogonal reader option, not part of strict versus lax token acceptance.

7.3.11 Source Notes

- Documented: the repository-wide AutoLISP specification draft records that integers are signed 32-bit values and that literal overflow may produce a real.
- Inferred: restrictive grammar chosen because the vendor corpus lacks a full formal reader grammar.
- Under-specified: broader lexical acceptance in real products.

7.4 Strict Real Literal

7.4.1 Name

Strict real literal

7.4.2 Class

Token Class

7.4.3 Concrete Syntax

```
[+-]?[0-9]+\\.[0-9]+([eE][+-]?[0-9]+)?
```

7.4.4 Constituents

- optional sign
- integer part
- decimal point
- fractional part
- optional exponent

7.4.5 Description

Portable decimal real token recognized in strict mode.

7.4.6 Acceptance Rules

- Entire token must match the grammar.
- At least one digit is required on both sides of the decimal point.
- Integer-shaped decimal tokens outside the signed 32-bit integer range are also read as real objects.

7.4.7 Strict/Lax Policy

- Strict mode rejects `.5`, `1.`, and `1e3`.
- Lax mode may accept those forms and other implementation-defined extensions.

7.4.8 Examples

- `0.0`
- `3.5`
- `-12.75e+2`

7.4.9 Compatibility

- Chosen as a deliberately narrow compatibility core.

7.4.10 Notes

- This entry is separate from the behavior of string conversion functions such as `atof`.
- The reader may therefore produce real objects from either explicit real notation or overflowed integer-shaped notation.

7.4.11 Source Notes

- Documented: the repository-wide AutoLISP specification draft records that integer literal overflow may produce a real.
- Inferred: restrictive grammar chosen because vendor materials do not yet settle the full lexical boundary.
- Under-specified: exact vendor acceptance of shorthand and exponent-only forms.

7.5 String Literal

7.5.1 Name

String literal

7.5.2 Class

Reader Syntax

7.5.3 Concrete Syntax

Double-quoted character sequence.

7.5.4 Constituents

- opening double quote
- string body
- closing double quote

7.5.5 Description

Represents a textual literal value.

7.5.6 Acceptance Rules

- Unterminated strings are syntax errors.
- Strict mode guarantees doubled double quote handling.
- Additional escape handling is implementation-defined unless established by later evidence.

7.5.7 Strict/Lax Policy

- Strict mode is deliberately conservative.
- Lax mode may accept broader escape forms under explicit policy.

7.5.8 Examples

- `"hello"`
- `"a ""quoted"" word"`

7.5.9 Compatibility

- Exact escape repertoire still requires further validation.

7.5.10 Notes

- String objects store both decoded contents and original source spelling.

7.5.11 Source Notes

- Documented: string literals are fundamental AutoLISP syntax.
- Under-specified: complete escape repertoire.
- Implementation-defined: initial strict escape subset beyond doubled quote.

8 Source Notes

- Documented: repository-wide architecture documents require a dedicated AutoLISP reader, explicit dialect handling, source-aware reader results, and separation from Common Lisp reader/package semantics.
- Documented: Autodesk concept and tutorial pages cover comments, symbols, strings, dotted pairs, and general AutoLISP language structure.
- Inferred: the concrete strict numeric regexes and the dual storage of original and canonical symbol/string data are conservative `clautolisp` design choices.
- Implementation-defined: canonical symbol case, recoverable malformed-node strategy in lax tooling mode, and any lax escape extensions.
- Implementation-defined: canonical symbol case, recoverable malformed-node strategy in tooling-oriented recovery mode, and any lax escape extensions.
- Under-specified: exact full vendor reader grammar, exact string escape repertoire, and the full lexical acceptance boundary of numeric spellings in real products.